AI ARCHITECTURE

RAG

VS

CAG

VS

MAG

Three augmentation paradigms.

Which one does your system actually need?

Naresh Edagotti Follow | PracticAI

Repost

THE CORE PROBLEM

Why Augmentation Exists

LLMs are powerful but frozen. They know nothing beyond their training cutoff and nothing about your business.

REAL SCENARIO: SUPPORT BOT

- User asks: "What's the current pricing for the Pro plan?"
- LLM answers from 18-month old training data.
- Result: Incorrect info. Trust is gone.

STATIC MEMORY

Parametric knowledge is baked into weights at training time.

AMNESIA

Models forget everything between sessions.

Same problem. Three different philosophies.

PARADIGM 1

RAG: How It Works

Dynamic retrieval at query time from external vector store.

1. Embed Query
Convert to vector representation

2. Vector DB Search
Cosine similarity, fetch top-k chunks

3. Prompt Assembly & LLM
Query + retrieved chunks, generate grounded answer

WHY

LLM alone can't access proprietary, evolving knowledge.

WHEN

Large corpus, varied queries, frequently updated docs.

Frameworks: LangChain, LlamaIndex, Haystack

Naresh Edagotti Follow | PracticAI

Repost

EVALUATION

RAG: Pros & Cons

STRENGTHS

- ✓ Scales to millions of documents (size is not a bottleneck)
- ✓ Always retrieves the freshest version of a document
- ✓ Source attribution (can cite which document it came from)
- ✓ Works across heterogeneous knowledge (PDFs, wikis, DBs)

WEAKNESSES

- × Real-time retrieval latency and system complexity
- × Retrieval failures cascade (wrong chunk = wrong answer)
- × Chunking breaks context across paragraph boundaries
- × Multi-hop reasoning is hard across multiple retrievals

⚠ **Real Failure: Fails to synthesize 5 chunks for full competitor pricing picture.**

Naresh Edagotti Follow | PracticAI

Repost

PARADIGM 2

CAG: How It Works

Preload all knowledge into context, cache KV states, answer directly.

1. Load Docs (Startup)
Load all documents into LLM context window

2. Precompute KV Cache
Save attention states to memory (done once)

3. Query Time (Fast Path)
Append query to cache, generate (no retrieval step)

WHY

Eliminates retrieval latency, errors, and infra complexity.

WHEN

Stable, bounded knowledge base that fits in context window.

Frameworks: vLLM, HF Transformers, Anthropic prompt caching

Naresh Edagotti Follow | PracticAI Repost

EVALUATION

CAG: Pros & Cons

STRENGTHS

- ✓ Reduced latency (eliminates real-time retrieval)
- ✓ No chunking errors (full documents are in context)
- ✓ Enhanced multi-hop reasoning (processes all info upfront)
- ✓ Simpler architecture (no embedding pipeline, no vector DB)

WEAKNESSES

- × Limited context window for extremely large datasets
- × Stale cache problem (requires full rebuild if knowledge changes)
- × Initial overhead to precompute KV cache
- × Not multi-tenant (one shared context)

⚠ **Real Failure: Answers with old pricing on Monday because it's running Sunday's cache.**

PARADIGM 3

MAG: How It Works

Dynamic evolving memory that persists, updates, and improves across sessions.

1. Current Session
Working memory + episodic/semantic memory retrieval

2. Memory Update & Consolidation
Important facts/preferences written back to memory store

3. Next Session Continuity
Memory retrieved for same user, providing full continuity

WHY

RAG and CAG are stateless. MAG gives continuity and personalization.

WHEN

Long-running agents, personal assistants, multi-session systems.

Frameworks: Mem0, Zep, MemGPT, LangGraph

Naresh Edagotti Follow | PracticAI Repost

EVALUATION

MAG: Pros & Cons

STRENGTHS

- ✓ True continuity (remembers preferences, decisions)
- ✓ Self-improving (gets richer the more the system is used)
- ✓ Personalization at scale (each user has evolving memory layer)
- ✓ Combines multiple memory types (episodic, semantic, procedural)

WEAKNESSES

- ✗ Memory staleness (old memories can contradict new facts)
- ✗ Privacy risk (stores contain sensitive personal information)
- ✗ Write strategy complexity (what to remember/forget is non-trivial)
- ✗ Highest infrastructure complexity of the three

⚠ **Real Failure: Confuses old and new cities if memory isn't updated properly.**

COMPARISON

Head to Head

SCENARIO 1: TELECOM SUPPORT BOT

10M customers, 500 product docs updating weekly.

Winner: RAG

Corpus too large for CAG. Queries vary wildly.

SCENARIO 2: HR POLICY CHATBOT

80 stable policy docs, 200K tokens total.

Winner: CAG

Docs fit in context. Policies don't change daily. Fast.

SCENARIO 3: EXECUTIVE ASSISTANT

Personal AI assistant used daily.

Winner: MAG

Remembers goals, preferences, decisions, style.

</> IMPLEMENTATION

Practical Code: RAG

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI
# Build vector store from docs
vectorstore = Chroma.from_documents(
    documents=docs,
    embedding=OpenAIEmbeddings()
)
# RAG chain - retrieves at every query
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5})
)
response = qa_chain.run("What is the refund policy?")
```

- Retrieves fresh docs on every query
- Scales to any corpus size
- Add reranker between retriever and LLM for better precision

</> IMPLEMENTATION

Practical Code: CAG

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
# STEP 1: Preload all knowledge - done once at startup
inputs = tokenizer(knowledge, return_tensors="pt")
# Precompute KV cache - one-time cost
with torch.no_grad():
    outputs = model(**inputs, use_cache=True)
    kv_cache = outputs.past_key_values  # save this
# STEP 2: Answer queries - reuse cache every time
def answer(query):
    query_tokens = tokenizer(query, return_tensors="pt")
    with torch.no_grad():
        result = model(
            **query_tokens,
            past_key_values=kv_cache  # reuse precomputed cache
        )
    return tokenizer.decode(result.logits.argmax(-1)[0])
```

- Model only processes new query tokens
- Full knowledge base fed into model once, saving attention states

Naresh Edagotti Follow | PracticAI Repost

</> IMPLEMENTATION

Practical Code: MAG

```
from mem0 import Memory
# Initialize memory layer
memory = Memory()
user_id = "user_naresh_001"
# STEP 1: Add memory from past interaction
memory.add("User prefers bullet point summaries.", user_id=user_id)
# STEP 2: On new session - retrieve relevant memories
def generate_with_memory(query, user_id):
    past_memories = memory.search(query, user_id=user_id)
    memory_context = "\n".join([m["memory"] for m in past_memories])

    prompt = f"Context:\n{memory_context}\nQuery: {query}"
    return llm.invoke(prompt)
# STEP 3: After session - update memory
memory.add("User is now exploring CAG and MAG.", user_id=user_id)
```

- Memory persists across sessions, updates over time, personalizes
- Mem0, Zep, and LangGraph are production-ready options

SUMMARY

Decision Framework

1. CORPUS SIZE?

- Fits in context window: **CAG**
- Too large for context: **RAG / MAG**

2. CHANGE FREQ?

- Stable for months: **CAG**
- Changes daily: **RAG**
- Evolves with user: **MAG**

3. MULTI-SESSION?

- Yes, users return: **MAG**
- No, independent: **RAG / CAG**

4. PRIORITY?

- Speed / low latency: **CAG**
- Always-fresh info: **RAG**
- Personalized context: **MAG**

HIGH-PERFORMANCE SYSTEMS

Hybrid Tiered: CAG (Core Identity) + RAG (Long-Tail Search)

Naresh Edagotti Follow | PracticAI Repost